

Exercise 1: Singleton Pattern — Logger

Logger.java

```
public class Logger {
    private static Logger instance;
    private Logger() {}

    public static Logger getInstance() {
        if (instance == null) {
            instance = new Logger();
        }
        return instance;
    }

    public void log(String message) {
        System.out.println("[LOG] " + message);
    }
}
```

SingletonTest.java

```
public class SingletonTest {
    public static void main(String[] args) {
        Logger logger1 = Logger.getInstance();
        Logger logger2 = Logger.getInstance();
        logger1.log("First log message");
        logger2.log("Second log message");
        System.out.println(logger1 == logger2); // true
    }
}
```

Exercise 2: Factory Method Pattern — Documents

Document.java (Interface)

```
public interface Document {  
    void open();  
}
```

Concrete Documents

```
public class WordDocument implements Document {  
    public void open() { System.out.println("Opening Word document"); }  
}
```

```
public class PdfDocument implements Document {  
    public void open() { System.out.println("Opening PDF document"); }  
}
```

```
public class ExcelDocument implements Document {  
    public void open() { System.out.println("Opening Excel document"); }  
}
```

DocumentFactory.java

```
public abstract class DocumentFactory {  
    public abstract Document createDocument();
```

```
    public void openDocument() {  
        Document doc = createDocument();  
        doc.open();  
    }  
}
```

```
public class WordDocumentFactory extends DocumentFactory {  
    public Document createDocument() { return new WordDocument(); }  
}
```

```
public class PdfDocumentFactory extends DocumentFactory {  
    public Document createDocument() { return new PdfDocument(); }  
}
```

```
public class ExcelDocumentFactory extends DocumentFactory {
```

```
    public Document createDocument() { return new ExcelDocument(); }  
}
```

FactoryTest.java

```
public class FactoryTest {  
    public static void main(String[] args) {  
        DocumentFactory factory = new WordDocumentFactory();  
        factory.openDocument();  
  
        factory = new PdfDocumentFactory();  
        factory.openDocument();  
  
        factory = new ExcelDocumentFactory();  
        factory.openDocument();  
    }  
}
```

Exercise 3: Builder Pattern — Computer

Computer.java

```
public class Computer {
    private String cpu;
    private String ram;
    private String storage;
    private String gpu;

    private Computer(Builder builder) {
        this.cpu    = builder.cpu;
        this.ram    = builder.ram;
        this.storage = builder.storage;
        this.gpu    = builder.gpu;
    }

    @Override
    public String toString() {
        return "Computer{cpu=" + cpu + ", ram=" + ram
            + ", storage=" + storage + ", gpu=" + gpu + "}";
    }

    public static class Builder {
        private String cpu;
        private String ram;
        private String storage;
        private String gpu;

        public Builder cpu(String cpu)    { this.cpu = cpu;    return this; }
        public Builder ram(String ram)    { this.ram = ram;    return this; }
        public Builder storage(String storage) { this.storage = storage; return this; }
        public Builder gpu(String gpu)    { this.gpu = gpu;    return this; }
        public Computer build()           { return new Computer(this); }
    }
}
```

BuilderTest.java

```
public class BuilderTest {
    public static void main(String[] args) {
        Computer gaming = new Computer.Builder()
            .cpu("Intel i9")
    }
}
```

```
.ram("32GB")  
.storage("1TB SSD")  
.gpu("RTX 4090")  
.build();
```

```
Computer office = new Computer.Builder()  
    .cpu("Intel i5")  
    .ram("16GB")  
    .storage("512GB SSD")  
    .build();
```

```
System.out.println(gaming);  
System.out.println(office);  
}  
}
```

Exercise 4: Adapter Pattern — Payment Gateways

PaymentProcessor.java (Target Interface)

```
public interface PaymentProcessor {  
    void processPayment(double amount);  
}
```

Adaptee Classes (Third-Party Gateways)

```
public class PayPalGateway {  
    public void makePayment(double amount) {  
        System.out.println("PayPal payment: $" + amount);  
    }  
}
```

```
public class StripeGateway {  
    public void charge(double amount) {  
        System.out.println("Stripe charge: $" + amount);  
    }  
}
```

Adapter Classes

```
public class PayPalAdapter implements PaymentProcessor {  
    private PayPalGateway paypal;  
    public PayPalAdapter(PayPalGateway paypal) { this.paypal = paypal; }  
  
    public void processPayment(double amount) {  
        paypal.makePayment(amount);  
    }  
}
```

```
public class StripeAdapter implements PaymentProcessor {  
    private StripeGateway stripe;  
    public StripeAdapter(StripeGateway stripe) { this.stripe = stripe; }  
  
    public void processPayment(double amount) {  
        stripe.charge(amount);  
    }  
}
```

AdapterTest.java

```
public class AdapterTest {  
    public static void main(String[] args) {  
        PaymentProcessor pp = new PayPalAdapter(new PayPalGateway());  
        pp.processPayment(150.00);  
  
        PaymentProcessor sp = new StripeAdapter(new StripeGateway());  
        sp.processPayment(250.00);  
    }  
}
```

Exercise 5: Decorator Pattern — Notifier

Notifier.java (Interface)

```
public interface Notifier {  
    void send(String message);  
}
```

EmailNotifier.java (Concrete Component)

```
public class EmailNotifier implements Notifier {  
    public void send(String message) {  
        System.out.println("Email: " + message);  
    }  
}
```

NotifierDecorator.java + Concrete Decorators

```
public abstract class NotifierDecorator implements Notifier {  
    protected Notifier wrappee;  
    public NotifierDecorator(Notifier wrappee) { this.wrappee = wrappee; }  
    public void send(String message) { wrappee.send(message); }  
}
```

```
public class SMSNotifierDecorator extends NotifierDecorator {  
    public SMSNotifierDecorator(Notifier wrappee) { super(wrappee); }  
    public void send(String message) {  
        super.send(message);  
        System.out.println("SMS: " + message);  
    }  
}
```

```
public class SlackNotifierDecorator extends NotifierDecorator {  
    public SlackNotifierDecorator(Notifier wrappee) { super(wrappee); }  
    public void send(String message) {  
        super.send(message);  
        System.out.println("Slack: " + message);  
    }  
}
```

DecoratorTest.java


```
public class DecoratorTest {  
    public static void main(String[] args) {  
        Notifier notifier = new SlackNotifierDecorator(  
            new SMSNotifierDecorator(  
                new EmailNotifier()));  
        notifier.send("Server is down!");  
    }  
}
```

Exercise 6: Proxy Pattern — Image Viewer

Image.java (Interface)

```
public interface Image {  
    void display();  
}
```

ReallImage.java + ProxyImage.java

```
public class ReallImage implements Image {  
    private String filename;  
  
    public ReallImage(String filename) {  
        this.filename = filename;  
        loadFromServer();  
    }  
  
    private void loadFromServer() {  
        System.out.println("Loading " + filename + " from server...");  
    }  
  
    public void display() {  
        System.out.println("Displaying " + filename);  
    }  
}  
  
public class ProxyImage implements Image {  
    private String filename;  
    private ReallImage reallImage;  
  
    public ProxyImage(String filename) { this.filename = filename; }  
  
    public void display() {  
        if (reallImage == null) {  
            reallImage = new ReallImage(filename); // lazy init  
        }  
        reallImage.display();  
    }  
}
```

ProxyTest.java

```
public class ProxyTest {  
    public static void main(String[] args) {  
        Image img = new ProxyImage("photo.jpg");  
        img.display(); // loads then displays  
        img.display(); // displays from cache  
    }  
}
```

Exercise 7: Observer Pattern — Stock Market

Stock.java + Observer.java (Interfaces)

```
import java.util.List;

public interface Stock {
    void register(Observer o);
    void deregister(Observer o);
    void notifyObservers();
}

public interface Observer {
    void update(String stockName, double price);
}
```

StockMarket.java (Concrete Subject)

```
import java.util.ArrayList;
import java.util.List;

public class StockMarket implements Stock {
    private List<Observer> observers = new ArrayList<>();
    private String stockName;
    private double price;

    public void setPrice(String name, double price) {
        this.stockName = name;
        this.price = price;
        notifyObservers();
    }

    public void register(Observer o) { observers.add(o); }
    public void deregister(Observer o) { observers.remove(o); }
    public void notifyObservers() {
        for (Observer o : observers) o.update(stockName, price);
    }
}
```

MobileApp.java + WebApp.java + ObserverTest.java

```
public class MobileApp implements Observer {
```

```
    public void update(String name, double price) {  
        System.out.println("[Mobile] " + name + " = $" + price);  
    }  
}
```

```
public class WebApp implements Observer {  
    public void update(String name, double price) {  
        System.out.println("[Web] " + name + " = $" + price);  
    }  
}
```

```
public class ObserverTest {  
    public static void main(String[] args) {  
        StockMarket market = new StockMarket();  
        market.register(new MobileApp());  
        market.register(new WebApp());  
        market.setPrice("AAPL", 189.50);  
        market.setPrice("GOOG", 140.30);  
    }  
}
```

Exercise 8: Strategy Pattern — Payment Methods

PaymentStrategy.java + Concrete Strategies

```
public interface PaymentStrategy {
    void pay(double amount);
}

public class CreditCardPayment implements PaymentStrategy {
    private String cardNumber;
    public CreditCardPayment(String cardNumber) { this.cardNumber =
cardNumber; }
    public void pay(double amount) {
        System.out.println("Paid $" + amount + " via Credit Card: " + cardNumber);
    }
}

public class PayPalPayment implements PaymentStrategy {
    private String email;
    public PayPalPayment(String email) { this.email = email; }
    public void pay(double amount) {
        System.out.println("Paid $" + amount + " via PayPal: " + email);
    }
}
```

PaymentContext.java + StrategyTest.java

```
public class PaymentContext {
    private PaymentStrategy strategy;
    public PaymentContext(PaymentStrategy strategy) { this.strategy = strategy; }
    public void setStrategy(PaymentStrategy strategy) { this.strategy = strategy; }
    public void executePayment(double amount) { strategy.pay(amount); }
}

public class StrategyTest {
    public static void main(String[] args) {
        PaymentContext ctx = new PaymentContext(new CreditCardPayment("4111-
1111-1111-1111"));
        ctx.executePayment(200.00);

        ctx.setStrategy(new PayPalPayment("user@example.com"));
        ctx.executePayment(75.00);
    }
}
```

}

Exercise 9: Command Pattern — Home Automation

Command.java + Light.java (Receiver)

```
public interface Command {
    void execute();
}

public class Light {
    private String name;
    public Light(String name) { this.name = name; }
    public void turnOn() { System.out.println(name + " is ON"); }
    public void turnOff() { System.out.println(name + " is OFF"); }
}
```

Concrete Commands + RemoteControl.java + CommandTest.java

```
public class LightOnCommand implements Command {
    private Light light;
    public LightOnCommand(Light light) { this.light = light; }
    public void execute() { light.turnOn(); }
}

public class LightOffCommand implements Command {
    private Light light;
    public LightOffCommand(Light light) { this.light = light; }
    public void execute() { light.turnOff(); }
}

public class RemoteControl {
    private Command command;
    public void setCommand(Command command) { this.command = command; }
    public void pressButton() { command.execute(); }
}

public class CommandTest {
    public static void main(String[] args) {
        Light living = new Light("Living Room");
        RemoteControl remote = new RemoteControl();

        remote.setCommand(new LightOnCommand(living));
        remote.pressButton();
    }
}
```



```
        remote.setCommand(new LightOffCommand(living));
        remote.pressButton();
    }
}
```

Exercise 10: MVC Pattern — Student Records

Student.java (Model)

```
public class Student {
    private String name;
    private int id;
    private String grade;

    public Student(String name, int id, String grade) {
        this.name = name; this.id = id; this.grade = grade;
    }

    public String getName() { return name; }
    public int getId() { return id; }
    public String getGrade() { return grade; }
    public void setGrade(String grade) { this.grade = grade; }
}
```

StudentView.java (View) + StudentController.java (Controller)

```
public class StudentView {
    public void displayStudentDetails(String name, int id, String grade) {
        System.out.println("Student: " + name + " | ID: " + id + " | Grade: " + grade);
    }
}

public class StudentController {
    private Student model;
    private StudentView view;

    public StudentController(Student model, StudentView view) {
        this.model = model; this.view = view;
    }

    public void setStudentGrade(String grade) { model.setGrade(grade); }
    public void updateView() {
        view.displayStudentDetails(model.getName(), model.getId(), model.getGrade());
    }
}
```

MVCTest.java

```
public class MVCTest {  
    public static void main(String[] args) {  
        Student student = new Student("Alice", 101, "B");  
        StudentView view = new StudentView();  
        StudentController controller = new StudentController(student, view);  
  
        controller.updateView();  
        controller.setStudentGrade("A");  
        controller.updateView();  
    }  
}
```

Exercise 11: Dependency Injection — Customer Management

CustomerRepository.java (Interface)

```
public interface CustomerRepository {  
    String findCustomerById(int id);  
}
```

CustomerRepositoryImpl.java

```
public class CustomerRepositoryImpl implements CustomerRepository {  
    public String findCustomerById(int id) {  
        // Simulates a DB lookup  
        if (id == 1) return "Alice";  
        if (id == 2) return "Bob";  
        return "Unknown";  
    }  
}
```

CustomerService.java

```
public class CustomerService {  
    private final CustomerRepository repository;  
  
    // Constructor injection  
    public CustomerService(CustomerRepository repository) {  
        this.repository = repository;  
    }  
  
    public void printCustomer(int id) {  
        String name = repository.findCustomerById(id);  
        System.out.println("Customer #" + id + ": " + name);  
    }  
}
```

DITest.java

```
public class DITest {  
    public static void main(String[] args) {  
        CustomerRepository repo = new CustomerRepositoryImpl();  
        CustomerService service = new CustomerService(repo);  
        service.printCustomer(1);  
    }  
}
```

```
        service.printCustomer(2);  
        service.printCustomer(3);  
    }  
}
```